

Resenha Crítica: Artigo 14 — Architectural Styles and the Design of Network-based Software Architectures (Fielding, 2000)

Gustavo Pessoa Firmino Duarte

23 de Abril de 2026

1 Introdução

A dissertação de doutorado de Roy T. Fielding, defendida em 2000 na Universidade da Califórnia em Irvine, é um dos documentos mais influentes da história da arquitetura de software para sistemas distribuídos. Nela, Fielding define e justifica o estilo arquitetural *Representational State Transfer* (REST), que se tornaria a base arquitetural da *World Wide Web* e, nas décadas seguintes, o modelo dominante para o design de APIs na indústria de software. O trabalho não é apenas uma proposta técnica: é uma metodologia para *avaliar* estilos arquiteturais através de suas propriedades e trade-offs, usando um rigoroso framework de análise que aplica e combina restrições arquiteturais para derivar propriedades de sistema.

2 REST: Restrições e Propriedades Derivadas

Fielding define REST como um conjunto de seis restrições arquiteturais, cada uma adicionando propriedades ao sistema e potencialmente removendo outras.

A restrição **cliente-servidor** separa as preocupações de interface do usuário das preocupações de armazenamento de dados, melhorando a portabilidade e a escalabilidade. A restrição **sem estado** (*stateless*) exige que cada requisição contenha toda a informação necessária para ser compreendida — o servidor não mantém sessão entre requisições. Isso melhora a visibilidade, a confiabilidade e a escalabilidade, mas aumenta o volume de dados enviados por requisição.

A restrição de **cache** permite que respostas sejam marcadas como cacheáveis ou não, melhorando a eficiência e a escalabilidade. A restrição de **interface uniforme** — a mais central e distintiva do REST — simplifica a arquitetura geral e melhora a visibilidade das interações ao custo de reduzir a eficiência para casos de uso específicos. Ela se desdobra em quatro subrestrições: identificação de recursos via URIs, manipulação de recursos através de representações, mensagens autodescritivas e *HATEOAS* (hipermídia como motor do estado da aplicação).

As restrições de **sistema em camadas** e **código sob demanda** (opcional) completam o conjunto, permitindo, respectivamente, a interposição de intermediários transparentes e a transferência de lógica client-side como JavaScript.

3 Conexão com a Prática Profissional

A API do meu MVP de e-commerce seguia REST de forma superficial: usava HTTP, verbos GET/POST/PUT/DELETE e URLs baseadas em recursos (`/products`, `/orders`). Mas ao ler Fielding, percebi que a restrição mais ignorada foi o *HATEOAS*: a API não retornava links para as transições de estado possíveis, exigindo que o cliente conhecesse de antemão toda a estrutura da API. Fielding argumenta que é exatamente o HATEOAS que permite a evolução do servidor sem quebrar clientes existentes — uma propriedade que, reconheço, teria sido valiosa caso o projeto tivesse evoluído além do MVP.

A restrição de ausência de estado também me fez refletir sobre a gestão de sessões de usuário: implementei autenticação via JWT armazenado no cliente, o que é RESTful no sentido de que o servidor não guarda estado de sessão — mas havia detalhes de implementação que criavam dependências implícitas de estado no servidor que violavam essa restrição.

Na ArcelorMittal, as integrações entre sistemas utilizavam desde APIs REST modernas até Web Services SOAP legados. A clareza do framework de Fielding é instruível para avaliar os trade-offs de cada abordagem.

4 Conclusão

A dissertação de Fielding é muito mais do que a definição do REST: é um exemplo exemplar de como avaliar e justificar escolhas arquiteturais através de um framework sistemático de restrições e propriedades derivadas. O REST tornou-se onipresente na indústria, mas frequentemente de forma superficial, ignorando restrições fundamentais como o *HATEOAS*. A leitura original de Fielding é essencial para compreender não apenas *o que* é REST, mas *por que* cada restrição existe e quais propriedades ela garante — um entendimento que permite avaliar criticamente as escolhas arquiteturais em qualquer projeto de API.